CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G2
System Integration and Architecture

System Design Document (SDD)

Project Title: Paluto

Prepared By: Auditor, Mechole Angelou M.

Version: 0.1

Date: February 23, 2026
Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	2-23-2026	Auditor, Mechole	Initial draft	Final
0.2	[Date]	[Your Name]	Added API specifications	Review
0.3	[Date]	[Your Name]	Updated database design	Review
0.4	[Date]	[Your Name]	Added UI/UX designs	Review
0.5	[Date]	[Your Name]	Incorporated feedback	Revised
0.6	[Date]	[Your Name]	Final review and corrections	Final
1	[Date]	[Your Name]	Baseline version for development	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY 4

1.0 INTRODUCTION 5

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION 5

3.0 NON-FUNCTIONAL REQUIREMENTS 8

4.0 SYSTEM ARCHITECTURE 9

5.0 API CONTRACT & COMMUNICATION 10

Authentication Endpoints 11

User Registration 11

User Login 11

6.0 DATABASE DESIGN 13

7.0 UI/UX DESIGN 14

8.0 PLAN 17

EXECUTIVE SUMMARY

1.1 Project Overview

Paluto is a specialized booking platform designed to bridge the gap between professional home cooks and busy families or professionals. Unlike standard food delivery apps, this system facilitates on-site meal preparation services, allowing customers to have personalized meals cooked directly in their own kitchens. The system includes a Spring Boot backend API, React web application, and Android mobile app, all integrated to provide a seamless appointment booking across platforms.

1.2 Objectives

1. Develop a fully functional Service-Booking platform with authentication, multi-user roles, cook portfolios, appointment booking scheduling, and service rating.
2. Implement a three-tier architecture using Spring Boot (backend), React (web), and Android (mobile)
3. Create RESTful APIs for communication between all system components
4. Design a responsive user interface that works consistently across web and mobile platforms
5. Deploy all system components to production-ready environments

1.3 Scope

Included Features:

- User registration and authentication (email/password)
- Google login integration
- Cook portfolio (add, remove, update certificates, menu, specialties, and rates)
- Transactional Appointment booking system
- Cook Service Rating System
- Payment Gateway Integration (sandbox)
- Email notification system
- In-app notifications

- Admin panel
- Native Android mobile application
- PostgreSQL database with relational schema

Excluded Features:

- Push notifications
- Advanced analytics dashboard

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for the Paluto Booking system. It provides detailed requirements, architectural decisions, API contracts, database design, and implementation roadmap to guide development and ensure all components integrate seamlessly.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: Paluto
Domain: Service-Booking platform
Primary Users:

1. Customers
2. Cooks
3. Administrator

Problem Statement: Busy professionals and families struggle to maintain healthy eating habits due to lack of time, often relying on unhealthy fast food.
Solution: A specialized booking platform that connects these individuals with home cooks for on-site meal preparations.

2.2 Core User Journeys

Journey 1: First-time customer booking a service

1. Customer opens the web or mobile application
2. Clicks "Sign Up" and creates an account

3. Customer logins with the registered account
4. Browse cook portfolios to view specialty menus and hourly rates
5. Selects a cook clicks “Book Appointment”
6. Selects specific date, time, and menu for the home cooking
7. Confirms the booking and provides the service address
8. Receives a booking confirmation once the cook accepts booking
9. After the service, the customer rates the cook.

Journey 2: Cook accepts a booking

1. Cook registers and completes their profile with certification/ specialties
2. Sets hourly rate and uploads specialty menus
3. Receives a notification for a new booking request
4. Navigates to “My bookings” tab
5. Click booking request and review details and confirm the schedule
6. After the service, the cook marks the appointment as completed.

Journey 3: Admin manages users

1. Admin logins into the web or mobile application
2. Navigates to “users” section
3. Click user details
4. View user details

2.3 Feature List (MoSCoW)

MUST HAVE

1. User authentication (register, login, logout)
2. Role based access (customer, cooks, admin)
3. Cook portfolio (certification, menus rates)
4. Appointment booking (date, time, menu, cook)

SHOULD HAVE

1. Cook Service rating
2. Booking history
3. Google login integration
4. Payment gateway integration

COULD HAVE

1. Favorite Cooks

2. Cook filtering
3. Cook dashboard
4. Email notification
5. In app notification
6. Responsive design for all screen sizes

WON'T HAVE

1. Real payment processing
2. Advanced analytics
3. Multi-language support
4. Real-time GPS tracking of the cook

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Customer Registration, Customer Login, Cook Registration, Cook Login, Forgot Password
- **Fields:** First name, Last name, address, phone number, Email, Password, Confirm Password
- **Validation:** Email format, password strength, uniqueness, phone number format,
- **API Endpoints:** POST /auth/register, POST /auth/login
- **Security:** JWT tokens, password hashing with bcrypt

Feature: Cook portfolio

- **Screens:** Portfolio View (Public), Portfolio edit (Cook)
- **Display:** Menu, rate, specialties, certification/s
- **Functions:** Add/edit menu, update rates, upload/ remove certifications.
- **API Endpoints:** GET /cook, GET /cook/{id}/ portfolio, GET /cook/portfolio/items, PUT /cook/portfolio/details

Feature: Appointment Booking

- **Screens:** Booking form, Appointment Summary, Booking History
- **Functions:** Select a cook, choose schedule (date & time), select menu, submit booking request, accept booking
- **Data Collected:** Service address, date/time, and menu.
- **API Endpoints:** POST /bookings, GET bookings/{id}, PUT /bookings/{id}/status

Feature: Service Rating

- **Screens:** Post-service Rating form, Cook profile rating section
- **Fields:** Star rating(1-5)

- **Persistence:** Ratings are linked to the cook's profile and appointment.
- **API Endpoints:** POST /ratings, GET /ratings/cook/{cookdId}

2.5 Acceptance Criteria

AC-1: Successful User Registration

- ➢ Given I am a new user
- ➢ When I enter valid firstname, lastname, phonenumber, address, email and strong password
- ➢ And confirm password matches
- ➢ And click "Create Account"
- ➢ Then my account should be created
- ➢ And I should be redirected to login page

AC-2: Cook Portfolio Creation

- ➢ Given I am logged in as a cook
- ➢ When I navigate to my portfolio dashboard and add/edit my specialty menus, hourly rates, and certifications.
- ➢ And I save the changes
- ➢ Then the updated information should be visible to customers when they search for cooks
- ➢ And the certifications should be stored and available for Admin verification.

AC-3: Appointment Booking Flow

- ➢ Given I am logged in as a "Customer"
- ➢ When I select a cook's profile and choose an available date and time slot.
- ➢ AndI provide the service location/address and confirm the booking request.
- ➢ Then the system should create a "Pending" appointment record.
- ➢ And the cook should see the new request in their management dashboard.

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time: ≤ 2 seconds for 95% of requests
- Web page load time: ≤ 3 seconds on broadband
- Mobile app cold start: ≤ 5 seconds
- Support 30 concurrent users
- Database queries complete within 500ms

3.2 Security Requirements

- HTTP for all communications
- JWT token authentication
- Password hashing with bcrypt
- Role-based access control
- SQL injection prevention
- XSS protection
- Rate limiting: 100 requests/minute per IP
- Admin endpoints require role verification

3.3 Compatibility Requirements

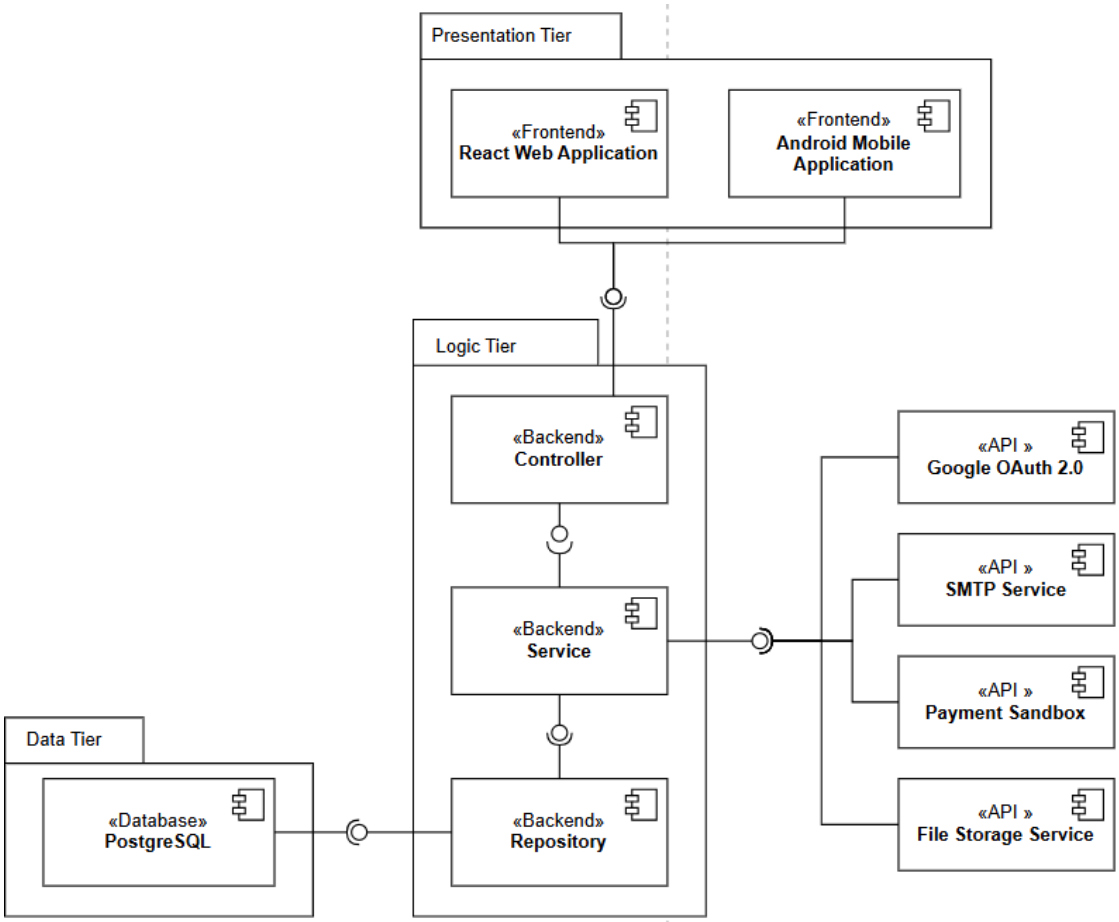
- **Web Browsers:** Chrome, Firefox, Safari, Edge (latest 2 versions)
- **Android:** API Level 24+ (Android 7.0+)
- **Screen Sizes:** Mobile (360px+), Tablet (768px+), Desktop (1024px+)

3.4 Usability Requirements

- New users should be able to complete their first cook booking within 5 minutes.
- Consistent navigation across all pages
- Clear error messages with recovery options

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** PostgreSQL 14+
- **Web Frontend:** React 18, TypeScript, Tailwind CSS, Axios
- **Mobile:** Kotlin, Retrofit, Room
- **Build Tools:** Maven (Backend), npm/yarn (Web), Gradle (Android)
- **Deployment:** Railway/Heroku (Backend), Vercel/Netlify (Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://[server_hostname]:[port]/api/
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header

Response Structure	{
	"success": boolean,
	"data": object|null,
	"error": {
	"code": string,
	"message": string,
	“details”: string,
	},
	"timestamp": string
	}

5.2 Endpoint Specifications

Authentication Endpoints

Customer Registration

Description	User Registration
API URL	/customer/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	“email”: <email>,
	“password”: <password>,
	“address”:<address>
	“firstname”: <firstname>,
	“lastname”: <lastname>,
	}

Response Structure	{
	"success": boolean,
	"data": {
	user {
	“email”: <email>,
	“firstname”: <firstname>,
	“lastname”: <lastname>,
	“address: <address>,
	“role”: “customer”,
	},
	“accessToken”: <token>,
	“refreshToken”: <token>,
	},
	"error": {
	"code": string,
	"message": string,

	“details”: string, }, "timestamp": string }

Cook Registration

Description	User Registration
API URL	/cook/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	“email”: <email>,
	“password”: <password>,
	“address”:<address>
	“firstname”: <firstname>,
	“lastname”: <lastname>,
	“hourly_rate”:<rate>,
	“years_xp”:<years>,
	“bio”:<bio>,
	}

Response Structure	{
	"success": boolean,
	"data": {
	user {
	“email”: <email>,
	“firstname”: <firstname>,
	“lastname”: <lastname>,
	“hourly_rate”: <rate>,
	“years_xp”: <years>,
	“bio”: <bio>
	“role”: “cook”,
	},
	“accessToken”: <token>,
	“refreshToken”: <token>,
	},
	"error": {
	"code": string,
	"message": string,
	“details”: string,
	},
	"timestamp": string
	}

User Login

Description	User Login
API URL	/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	“email”: <email>,
	“password”: <password>,
	}

Response Structure	{
	"success": boolean,
	"data": {
	“user”: {
	“email”: <email>,
	“firstname”: <firstname>,
	“lastname”: <lastname>,
	“address”:<address>,
	“role”: <role>,
	},
	“accessToken”: <token>
	},
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

Core Feature Endpoints

Cook Portfolio Update

Description	Update the cook portfolio details
API URL	/cook/portfolio/details
HTTP Method	PUT
Format	JSON for all requests/responses
Authentication	Bearer token (Role: cook)

Request Payload	{
	“bio”: <bio>,
	“years_xp”: <years>,
	“hourly_rate”: <rate>,
	}

Response Structure	{
	"success": boolean,
	"data": {
	"cook_id": <cook_id>,
	"updated_at": <datetime>
	},
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

Payment

Description	Get the payment URL from the gateway
API URL	/payments/checkout
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Bearer Token (Role: customer)

Request Payload	{
	"total_amount": <amount>,
	"customer_id": <customer_id>,
	}

Response Structure	{
	"success": boolean,
	"data": {
	"checkout_url": <url>,
	"reference": <reference>
	},
	},
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

Create a booking request

Description	Customer booking request creation with payment verification

API URL	/booking
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Bearer Token (Role: customer)

Request Payload	{
	“cook_id”: <cook_id>,
	“service_id”: <service_id>,
	“customer_id”: <customer_id>,
	“quantity”:<quantity>,
	“total_amount”: <total_amount>,
	“schedule_date”: <date>,
	“schedule_time”: <time>,
	“status”: “pending”,
	“reference”: <payment_gateway_id>,
	
	}

Response Structure	{
	"success": boolean,
	"data": {
	“booking_id”:<booking_id>,
	“payment_id”: <payment_id>
	},
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

Cook booking management

Description	Cook booking management and schedule conflict resolution
API URL	/booking/{id}/status
HTTP Method	PUT
Format	JSON for all requests/responses
Authentication	Bearer Token (Role: cook)

Request Payload	{
	“status”: <status>
	}

Response Structure	{
	"success": boolean,
	"data": {
	“booking_id”:<booking_id>,
	“status”: “accepted”

	},
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

Discovery Endpoints

Browse Cooks

Description	Retrieves a list of available cooks with optional filters
API URL	/cooks
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Bearer Token (Role: customer)
Request Payload	none

Response Structure	{
	"success": boolean,
	"data": [{
	"cook_id": <cook_id>,
	"firstname": <firstname>,
	"lastname": <lastname>,
	"bio": <bio>,
	"hourly_rate": <rate>,
	"years_xp": <years>,
	"rating_avg": <float>,
	}],
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

View the cook menu

Description	Fetches the specific menu or services offered by a selected cook.
API URL	/cooks/{id}/services
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Bearer Token (Any role)

Request Payload	none

Response Structure	{
	"success": boolean,
	"data": [{
	"service_id": <service_id>,
	"title": <title>,
	"description": <description>,
	“ingredient_ls: <ingredient_ls>
	"ingredeient_cost": <cost>,
	“est_prep_time”:<prep_time>,
	“image”:<image_url>,
	}],
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

Example 1	json
	{
	"success": false,
	"data": null,
	"error": {

	"code": "AUTH-001", "message": "Invalid credentials", "details": "Email or password is incorrect" }, "timestamp": "2024-01-28T10:30:00Z" }

Example 2	{
	"success": false,
	"data": null,
	"error": {
	"code": "VALID-001",
	"message": "Validation failed",
	"details": {
	"email": "Email is required",
	"password": "Must be at least 8 characters"
	}
	},
	"timestamp": "2024-01-28T10:30:00Z"
	}

Example 3	{
	"success": false,
	"data": null,
	"error": {
	"code": "BUS-001",
	"message": "Schedule Conflict",
	"details": “Cook is already booked in that schedule.”
	},
	"timestamp": "2024-01-28T10:30:00Z"
	}

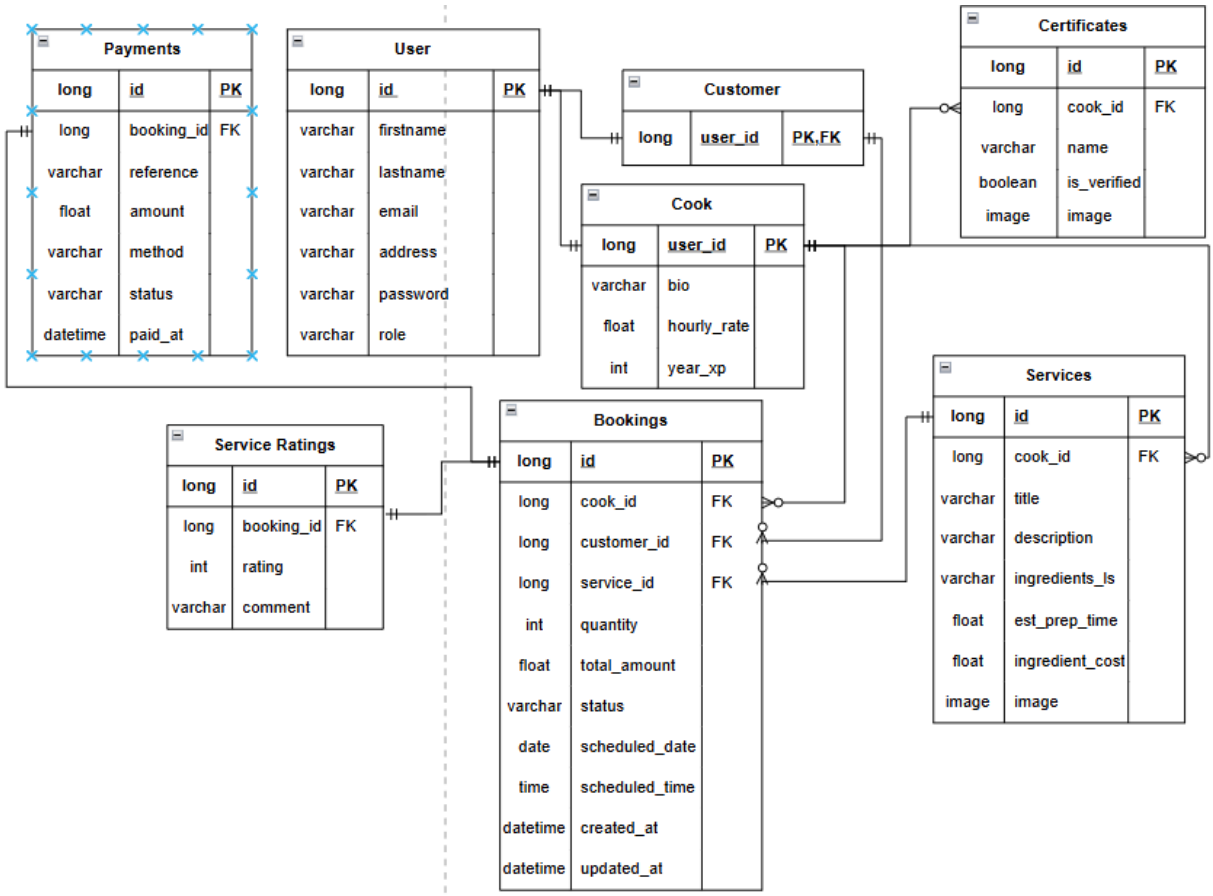
Example 4	{
	"success": false,
	"data": null,
	"error": {
	"code": "PAY-001",
	"message": "Payment Failed",
	"details": “Payment Reference invalid or unpaid.”
	},
	"timestamp": "2024-01-28T10:30:00Z"
	}

Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry
- BUS-001: Schedule Conflict
- PAY-001: Payment Failed
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



Detailed Relationships:

- **One-to-One:** User ↔ Cook (A cook is a user)
- **One-to-One:** User ↔ Customer (A customer is a user)
- **One-to-Many:** Cook → Service (A cook can offer multiple menu services)
- **One-to-Many:** Cook → Certificate (A cook can have multiple certificates)
- **One-to-Many:** Customer → Booking (A customer can have multiple bookings over time)
- **One-to-Many:** Cook → Booking (A cook can be assigned to multiple bookings)
- **One-to-Many:** Service → Booking (A specific service can be booked multiple times)
- **One-to-One:** Booking → Service Rating (A booking can only be rated one time)
- **One-to-One:** Booking → Payment (A booking can only have one payment)

Key Tables:

1. **users** - User accounts, authentication, and baseclass for customer/ cook
2. **cooks** - Extended profile for cooks
3. **customers** - Extended profile for customers
4. **services** - Specific offerings/ services of cooks (e.g., "Filipino breakfast")
5. **certificates** - Cook's personal certificates
6. **bookings** - Booking transactions
7. **service_ratings** - Rating of meal and service quality
8. **payments** - Records of payments for bookings

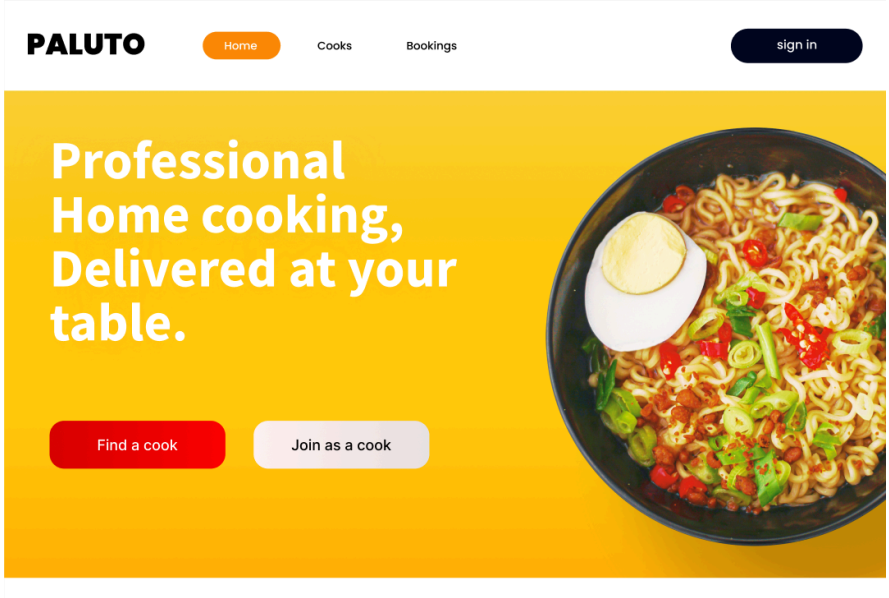
Table Structure Summary:

- **users:** id, email, firstname, lastname, address, password, role(customer/cook)
- **cooks:** user_id, bio, hourly_rate, years_exp
- **customers:** user_id,
- **services:** id, cook_id, title, description, ingredients_ls, est_prep_time, ingredients_cost, image
- **cook_certificates:** id, cook_id, cert_name, is_verified, image
- **bookings:** id, customer_id, cook_id, service_id,quantity, total_amount, status, scheduled_date, scheduled_time, created_at, updated_at
- **Service_ratings:** id, booking_id, rating, comment
- **payments** - id, bookind_id, reference, method, amount, status, paid_at

7.0 UI/UX DESIGN

7.1 Web Application Wireframes

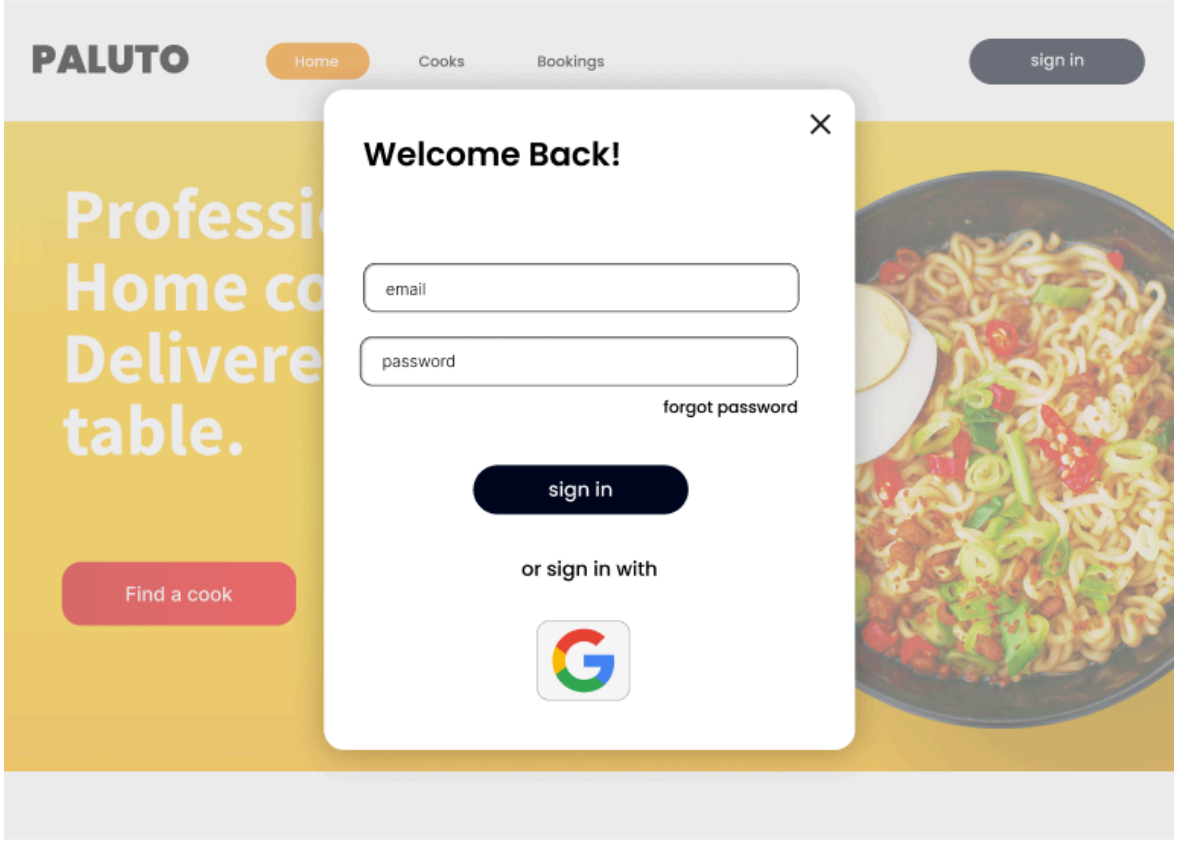
Landing page



Header: [Logo] [Nav bar] [“sign in” button]

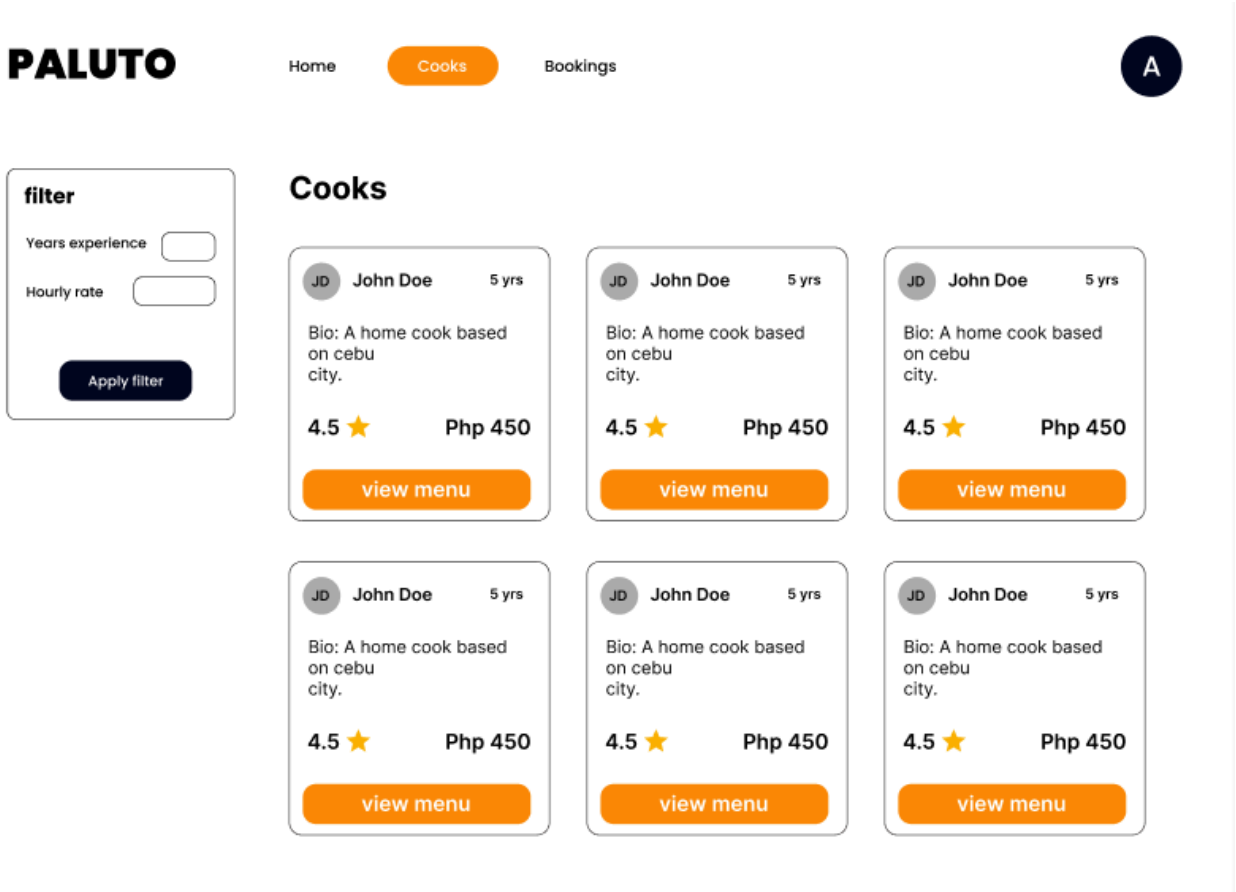
Main: Tagline, [“Find a cook” button] [“join as a cook” button]

Login Modal



Login form, Google button, Forgot password, Close button

Cooks page (List of cooks)

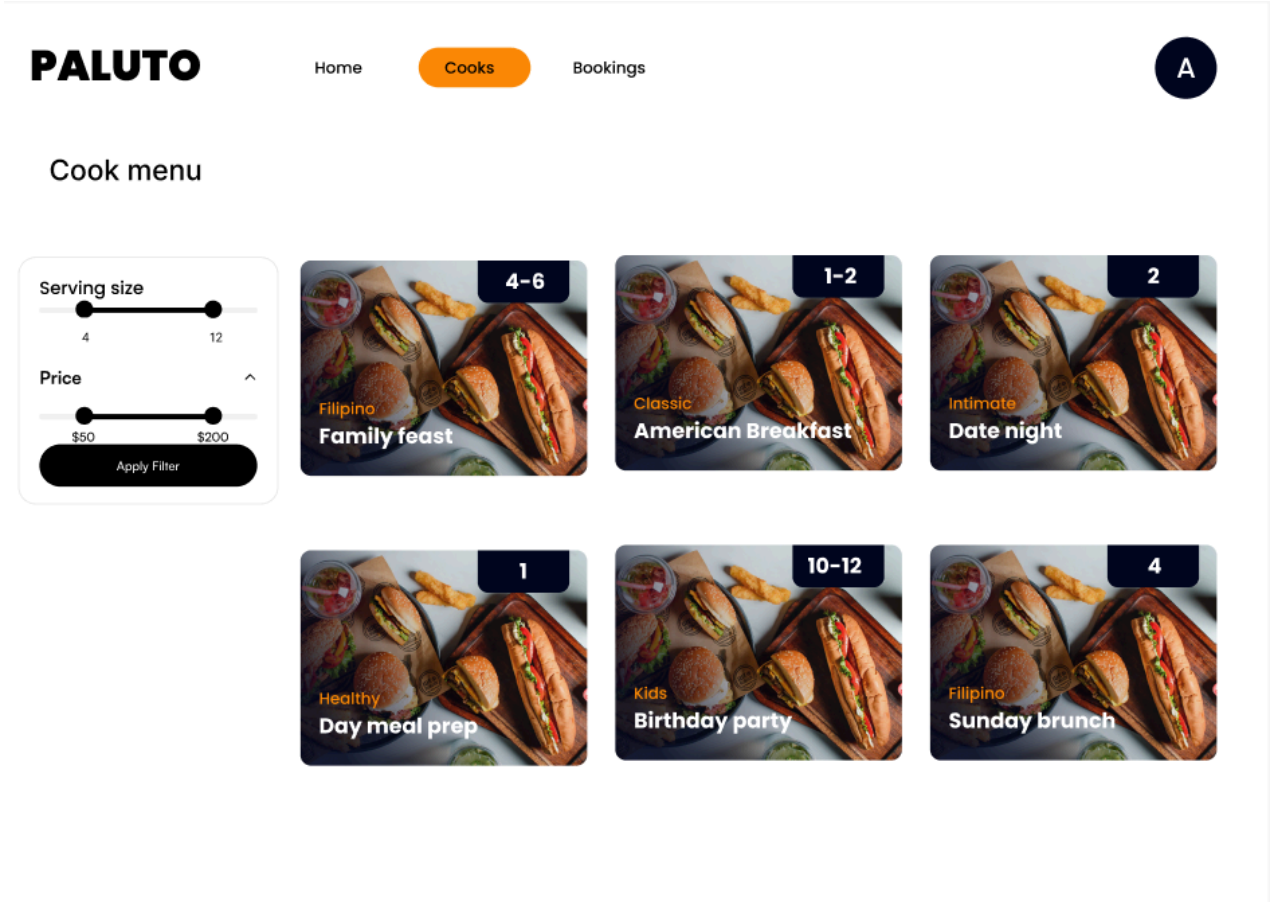


Header: [Logo] [Nav bar] [Profile]

Filter: Years of experience, hourly rate

Each cook card: Avatar, Full name, years of experience, Bio, rating, Hourly rate, “view menu” button

Cook's menu

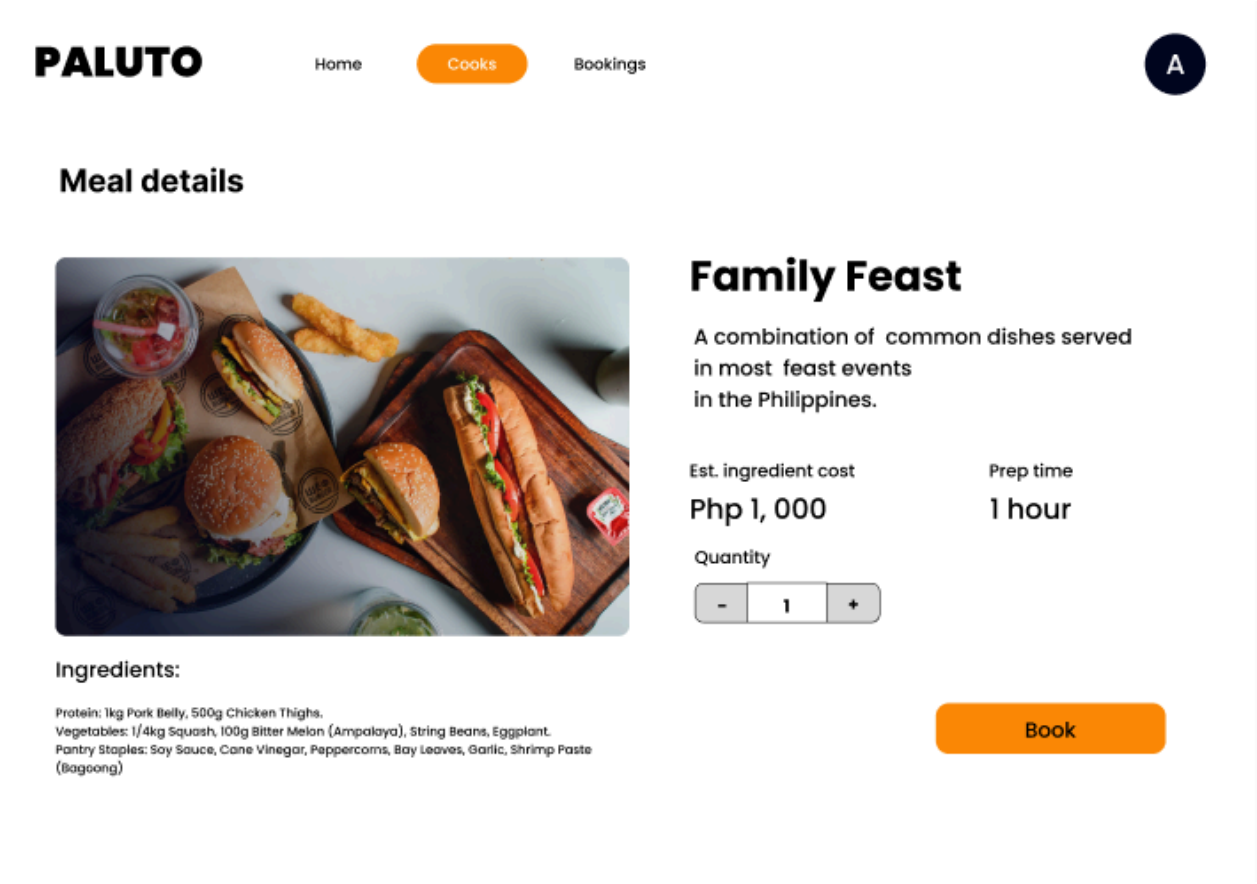


Header: [Logo] [Nav bar] [Profile]

Filter: Serving size, price

Each offering card: title, Serving size, offering image, title

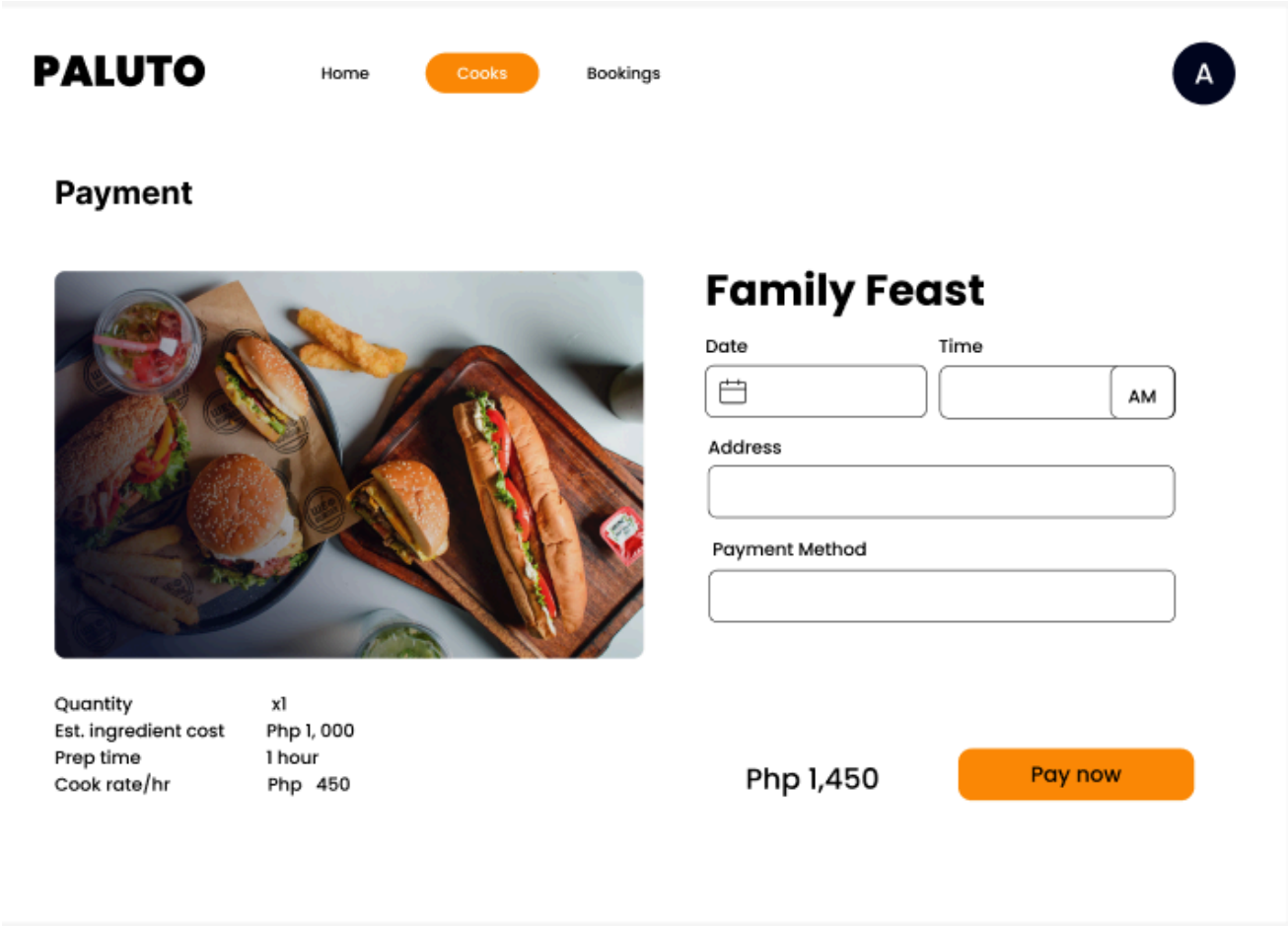
Meal details



Header: [Logo] [Nav bar] [Profile]

Meal image(Large), title, description, ingredients, ingredient cost, prep time, quantity stepper (default to 1), “Book” button

Payment page

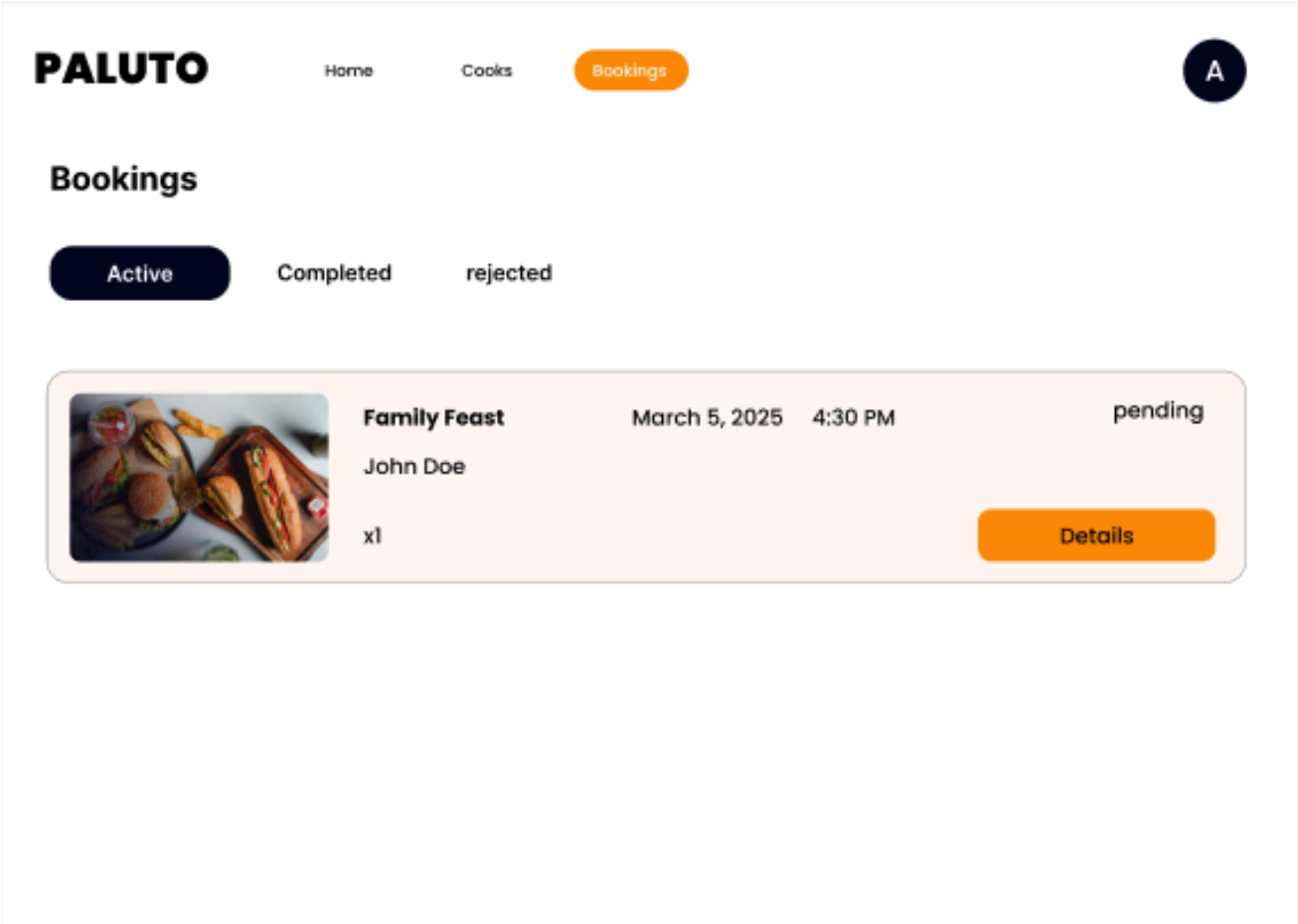


Header: [Logo] [Nav bar] [Profile]

Summary: Quantity, ingredient cost, prep time, cook's rate per hour

Title, Date picker, time picker, address field, payment method field, total cost, "Pay now" button

Bookings page (pending)

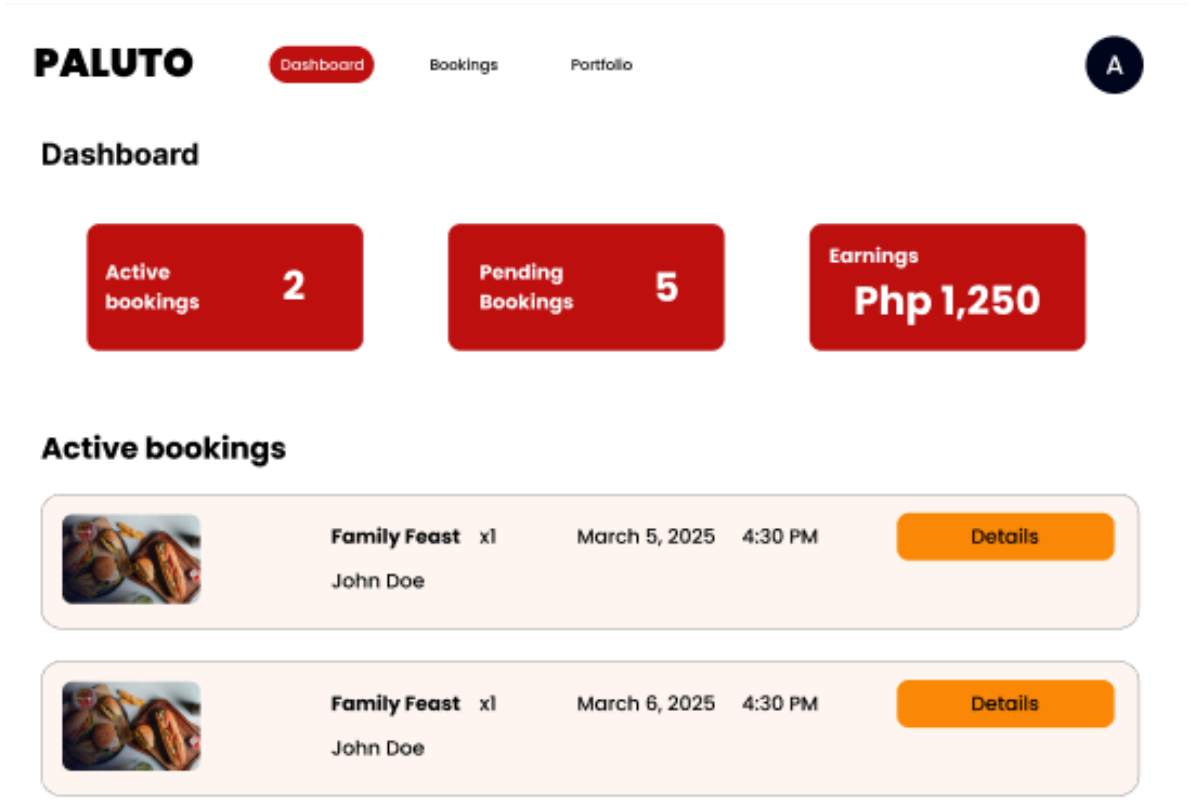


Header: [Logo] [Nav bar] [Profile]

Sub nav bar: Active, Completed, rejected

Each booking card: Meal image(small), title, cook's name, quantity, date, time, status, “Details” button.

Cook dashboard

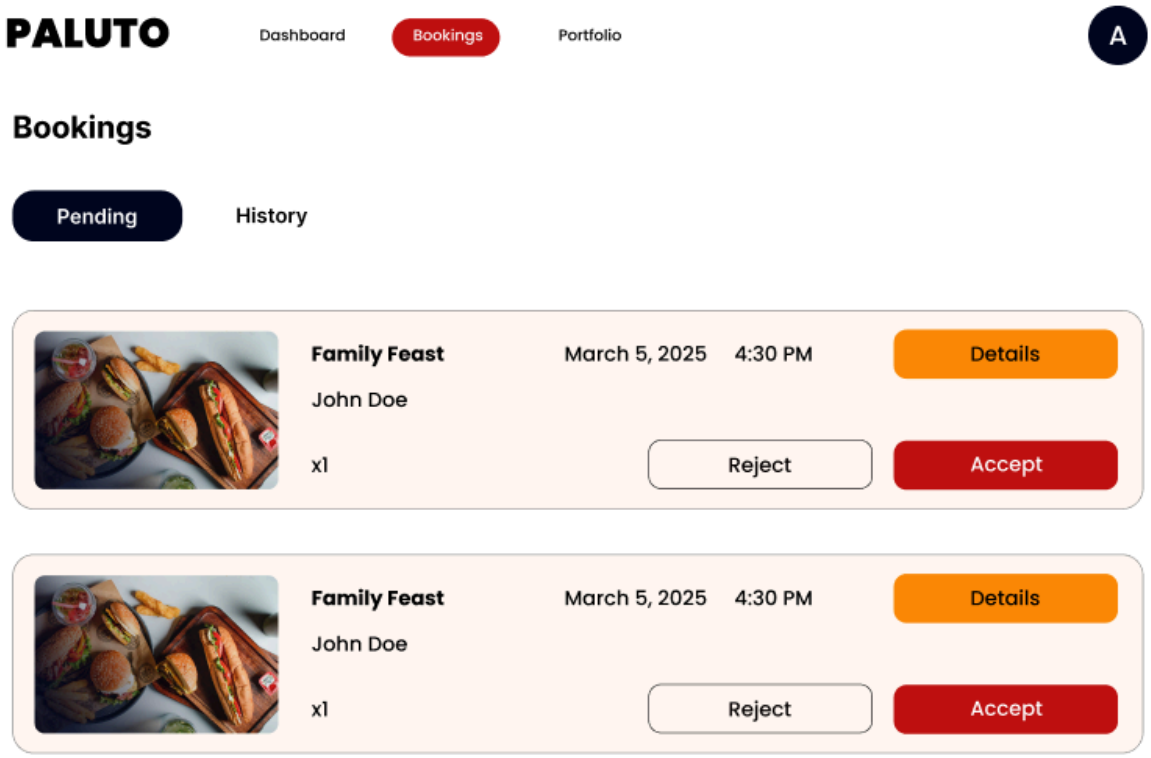


Header: [Logo] [Nav bar] [Profile]

KPI: Active bookings, Pending bookings, Earnings

Each active booking card: Meal image(small), title, quantity, customer name, date, time, “Details” button

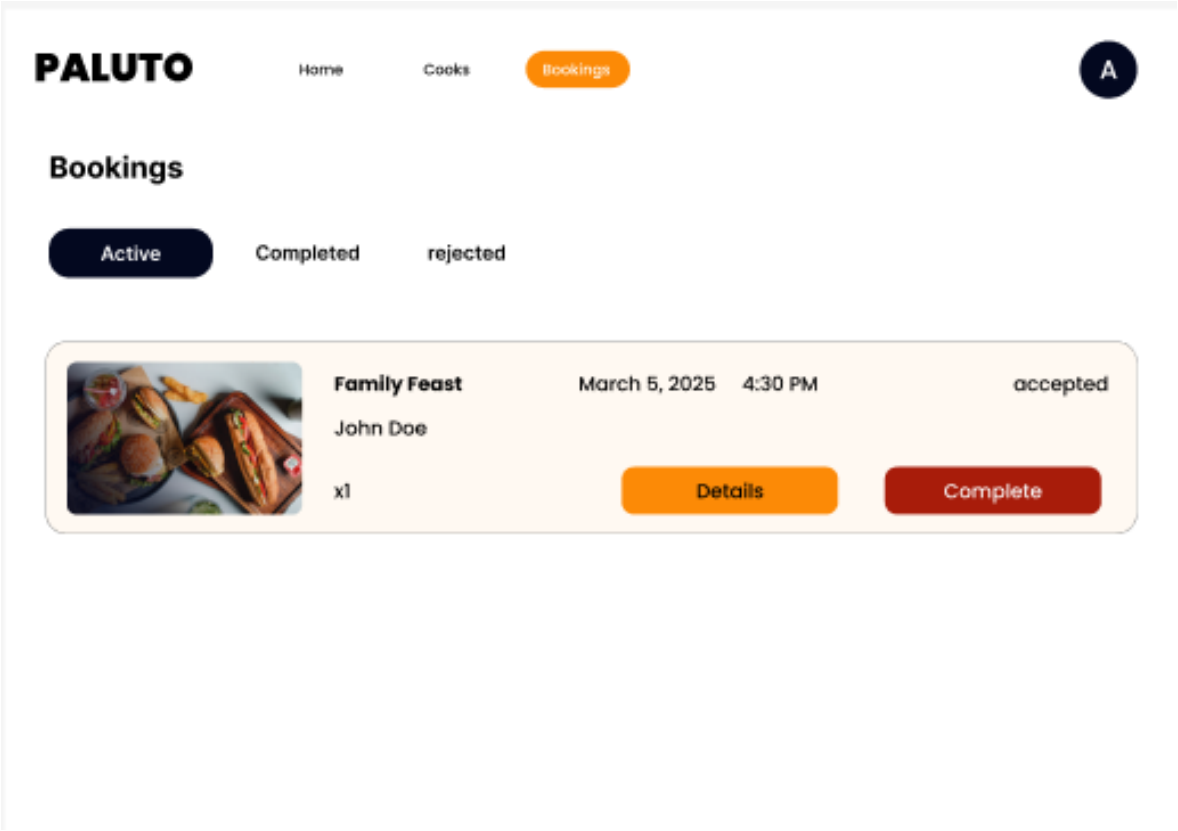
Cook bookings (pending)



Header: [Logo] [Nav bar] [Profile]

Each booking card: meal image(small), title, customer name, quantity, date, time, “details” button, “Reject” button, “Accept” button.

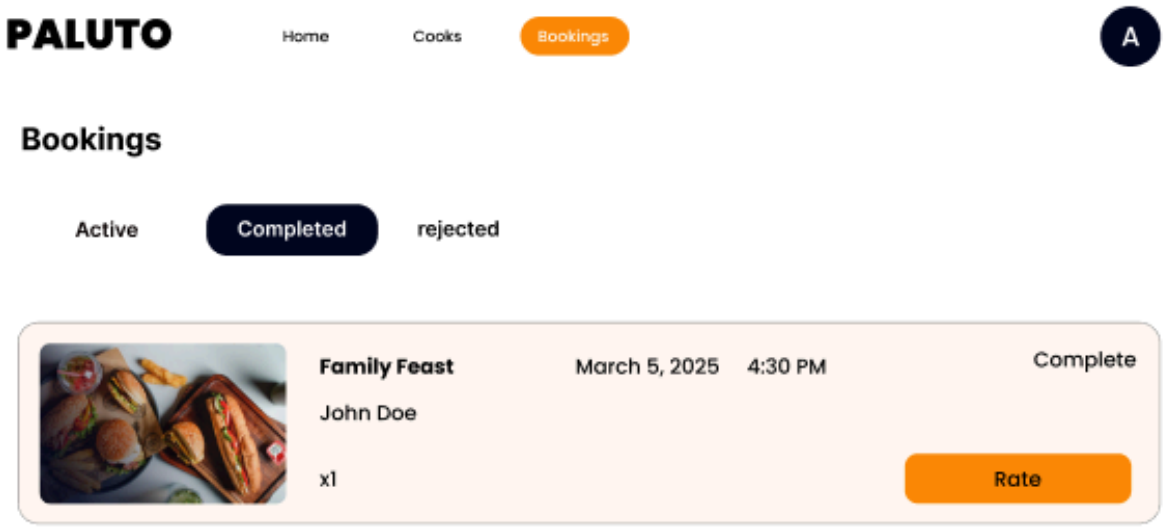
Customer bookings (accepted bookings)



Header: [Logo] [Nav bar] [Profile]

Each booking card: meal image(small), title, customer name, quantity, date, time, status, “details” button, “Complete” button

Customer bookings (completed bookings)



Header: [Logo] [Nav bar] [Profile]

Each booking card: meal image(small), title, customer name, quantity, date, time, status, “rate” button

7.2 Mobile Application Wireframes

Customer Bottom Navigation

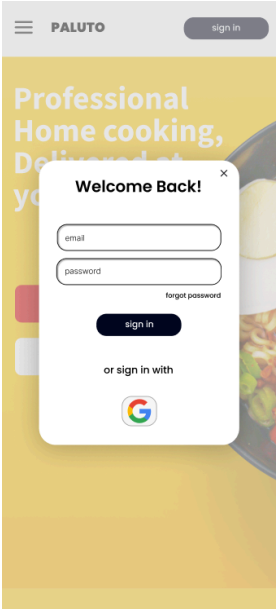
Items: Home, Cooks, Bookings

Landing screen

Header: Hamburger menu, Logo, “sign in” button

Main: “Find a cook” button, “Join as a cook” button

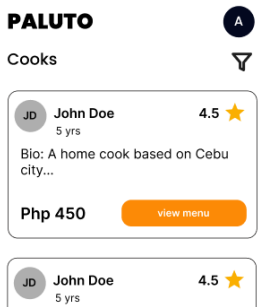
Login Modal



Fields: Email, password

Forgot password, “sign in” button, sign in with “Google” button

Cooks screen (List of cooks)



Header: Logo, Profile

Each cook card: Fullname, avatar, rating, bui, rate per hour, “view menu” button

filter icon, customer bottom navigation

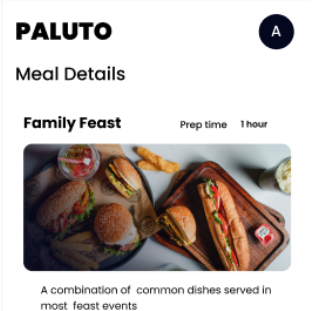
Cook’s menu

Header: Logo, Profile

Each menu card: title, serving size, offering image (background)

filter icon, bottom navigation

Meal details

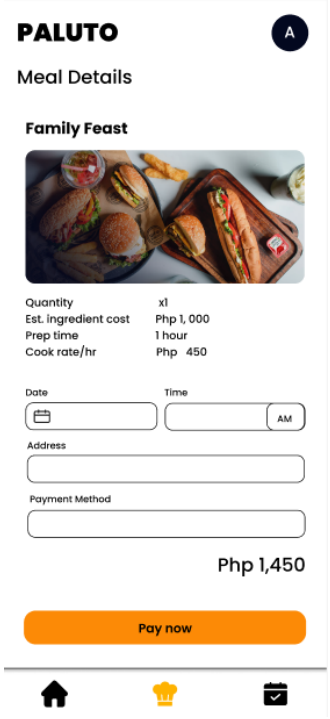


Header: Logo, Profile

title, prep time, meal image (medium), description, ingredients, ingredients cost, quantity stepper (Default to 1)

bottom navigation

Payment screen



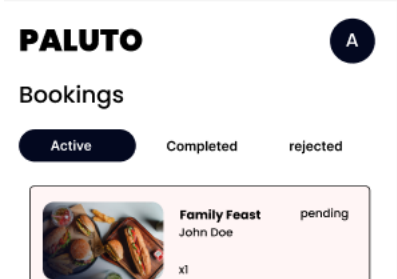
Header: Logo, Profile

Summary: Quantity, ingredient cost, prep time, cook's rate per hour

Title, Date picker, time picker, address field, payment method field, total cost, "Pay now" button

bottom navigation

Bookings screen (pending)



Header: Logo, Profile

Screen nav bar: Active, Completed, rejected

Each booking card: Meal image(small), title, cook's name, quantity, date, time, status, “Details” button.

Bottom navigation

Cook’s bottom navigation

Items: Dashboard icon, Bookings icon, Portfolio icon

Cooks dashboard

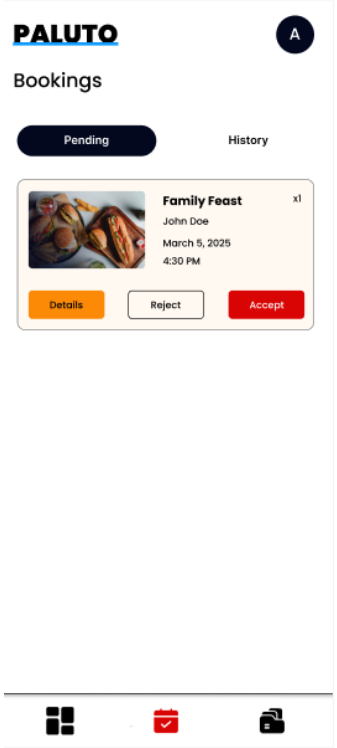
Header: Logo, Profile

KPI: Active bookings, Pending bookings, Earnings

Each active booking card: Meal image(small), title, quantity, customer name, date, time, “Details” button

Cook’s bottom navigation

Cooks bookings (pending)

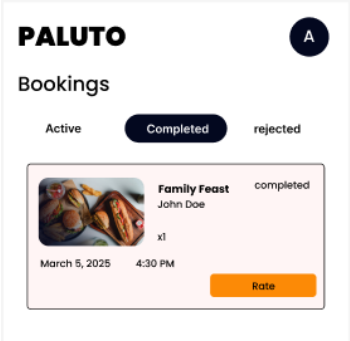


Header: Logo, Profile

Each booking card: meal image(small), title, customer name, quantity, date, time, “details” button, “Reject” button, “Accept” button.

Cook’s bottom navigation

Customer bookings (accepted)

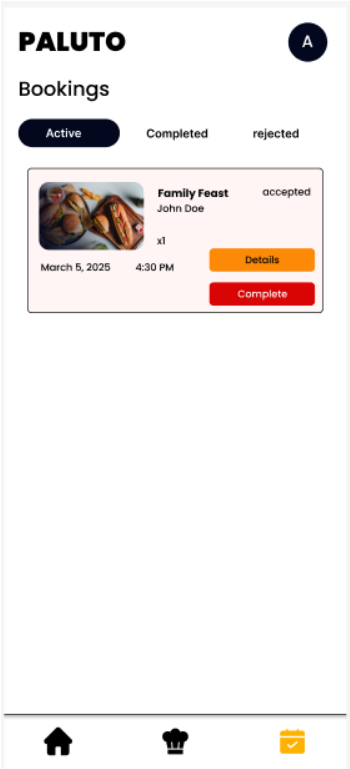


Header: Logo, Profile

Each booking card: meal image(small), title, customer name, quantity, date, time, status, “details” button, “Complete” button

Cook’s bottom navigation

Customer bookings (completed)



Header: Logo, Profile

Each booking card: meal image(small), title, customer name, quantity, date, time, status, “rate” button

Cook’s bottom navigation

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup and documentation

Day 3-4: Complete FRS and NFR

Day 5-7: System architecture design

Week 2: Detailed Design

Day 1-2: API specification

Day 3-4: Database design

Day 5-6: UI/UX wireframes

Day 7: Implementation plan finalization

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Spring Boot setup with dependencies

Day 2: Database configuration and entities

Day 3: JWT authentication & BCrypt password hashing implementation

Day 4: User management and /me endpoints

Day 5: Cook Portfolio CRUD operations

Week 4: Core Features

Day 1: Appointment/Booking transaction logic

Day 2: Service rating and file storage implementation

Day 3: Cooks and Menu filtering, Admin controls

Day 4: Error handling and validation

Day 5: API documentation and testing

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: React setup

Day 2: Authentication pages (login, register)

Day 3: Cook portfolio (menu, certs) and Cook listing page (Public view)

Day 4:Menu listing, menu details page

Day 5: Appointment booking form implementation

Week 6: Complete Web Features

Day 1: Appointment management (Accept/Decline flow for Cooks)

Day 2: Booking history, Status updates, and Rating system

Day 3: Admin dashboard for Cook certification moderation

Day 4: Responsive design polish

Day 5: API integration and End-to-End testing

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio setup and project structure

Day 2: Authentication screens (JWT Login/Register)

Day 3: Cook browsing and Portfolio view (XML Layouts)

Day 4: Appointment request and Calendar interaction

Day 5: Retrofit API service layer and JWT secure storage

Week 8: Complete Mobile App

Day 1: Role-aware UI

Day 2: Appointment and status management

Day 3: UI polish and role-based menu filtering

Day 4: Testing on emulator/device

Day 5: APK generation and documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: End-to-end testing across platforms

Day 2: Bug fixes and optimization

Day 3: Security review

Day 4: Performance testing

Day 5: Documentation updates

Week 10: Deployment

Day 1: Backend deployment (Railway/Heroku)

Day 2: Web app deployment (Vercel/Netlify)

Day 3: Mobile APK distribution

Day 4: Final testing

Day 5: Project submission

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend API (Auth & Booking) fully functional
- **M3 (End Week 6):** Web application complete
- **M4 (End Week 8):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Authentication system, Google OAuth login, and Role-based access control (Week 3)
2. Cook portfolio API (Week 3-4)
3. Booking Transaction Engine - Handling payments and status updates (Week 4)
4. Conflict Resolution Logic - Ensuring the cook can't accept a booking with a similar timeline(Week 4)
5. Role-Aware Mobile Navigation
6. Cross-platform testing (Week 9)

Risk Mitigation:

- Start with the simplest working version of each feature
- Test integration points early and often
- Keep a backup of the working versions
- Focus on core functionality before enhancements
- Database Safety First
- Limit External API Scope